EE 485 Term Project: House Price Prediction Using Machine Learning

Final Report: Comprehensive Model Comparison

Arda Bayam - [student ID removed]
Özgür Ekin Sönmez - [student ID removed]

December 21, 2025

Abstract

This report presents a comprehensive study on predicting house prices in Istanbul using three distinct machine learning algorithms: Ridge Regression, Neural Networks, and Decision Trees. All models and the data processing pipeline were implemented from scratch using NumPy, without reliance on high-level machine learning libraries. The study addresses the challenges of real estate price estimation, including high dimensionality, non-linearity, and the presence of outliers. Our experimental results demonstrate that the Neural Network model achieved the best performance with an R^2 score of 0.723 and a Mean Absolute Percentage Error (MAPE) of 33.30%, outperforming Ridge Regression (R^2 = 0.639) and the Decision Tree (R^2 = 0.629). This report details the mathematical foundations, implementation details, and comparative analysis of these methods.

1 Problem Description

The objective of this project is to develop a predictive model for estimating residential property prices in Istanbul based on a set of structural and locational attributes. The problem is formulated as a supervised regression task where the target variable is the listing price, and the input features include attributes such as district, number of rooms, floor level, building age, and heating type.

Real estate valuation is inherently complex due to the subjective nature of pricing, the influence of unobserved variables, and the presence of significant outliers. A robust solution requires not only effective modeling techniques but also a rigorous data preprocessing pipeline to handle missing values, encode categorical variables, and normalize numerical features.

We evaluated three modeling approaches to address this problem:

- **Ridge Regression:** A linear model incorporating L2 regularization to mitigate overfitting in high-dimensional feature spaces.
- **Neural Network:** A non-linear model capable of capturing complex interactions between features through hidden layers and activation functions.
- **Decision Tree:** A non-parametric model that recursively partitions the feature space to minimize prediction error.

2 Review of Implemented Methods

We implemented three fundamental machine learning algorithms from first principles to solve the regression problem.

- **Ridge Regression:** This method extends ordinary least squares (OLS) by adding a penalty term proportional to the square of the magnitude of coefficients. This regularization technique is particularly effective in dealing with multicollinearity and preventing overfitting. We derived and implemented the closed-form solution for the optimal weights.
- **Neural Network:** We constructed a Multi-Layer Perceptron (MLP) with two hidden layers (128, 64, and 32 neurons). The network utilizes the Rectified Linear Unit (ReLU) activation function to introduce non-linearity and is trained using mini-batch gradient descent with backpropagation. This approach allows the model to learn hierarchical representations of the input data.
- **Decision Tree:** We implemented a regression tree that recursively splits the data based on feature thresholds to minimize the Mean Squared Error (MSE) within each leaf node. The algorithm employs a greedy strategy to select the optimal split at each step, balancing model complexity with predictive accuracy through depth constraints.

The implementation also encompasses a complete data processing pipeline:

1. **Data Preparation:** Custom functions for parsing raw data, handling missing values, and removing outliers using the Interquartile Range (IQR) method.
2. **Feature Engineering:** Generation of polynomial features to capture non-linear relationships and one-hot encoding for categorical variables.
3. **Evaluation:** A rigorous comparison framework using metrics such as MSE, R^2, MAPE, MPE, and NMSE.

3 Dataset Description

The study utilizes the "House Price Dataset" sourced from Kaggle [1], comprising thousands of real estate listings from Istanbul.

3.1 Feature Overview

The dataset includes a diverse set of features that were processed for modeling:

- **Numerical Features:**
  - `NumberOfRooms`: Total room count, derived from parsing string representations (e.g., "3+1").
  - `GrossSquareMeters`: Total area of the property.
  - `NetSquareMeters`: Usable living area.
  - `BuildingAge`: Age of the structure, a critical factor for valuation.
- **Categorical Features:**
  - `district`: Administrative district, serving as a proxy for location value.
  - `FloorLocation`: Vertical position of the unit within the building.
  - `HeatingType`: Type of heating system.
  - `ItemStatus`: Furnishing status.

3.2 Target Variable

The target variable is `price`, representing the listing price in Turkish Lira (TL). The distribution of prices is right-skewed, necessitating robust outlier detection and removal strategies.

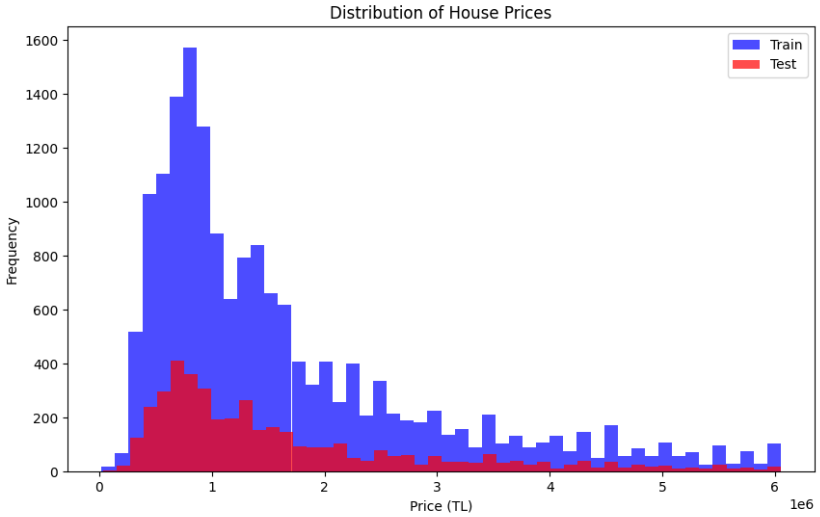


Figure 1: Distribution of House Prices (Train and Test Sets)

4 Data Preparation Methodology

Effective data preparation is crucial for model performance. Our pipeline includes the following stages:

4.1 Data Cleaning

Raw data required significant cleaning to be suitable for numerical analysis:

- **Parsing:** Removal of non-numeric characters (currency symbols, units) and conversion to floating-point format.
- **Feature Extraction:** Complex strings representing room counts were parsed and aggregated into single numerical values.
- **Standardization:** Categorical variations were unified, and age ranges were converted to representative numerical values.

4.2 Outlier Removal

To enhance model stability, we applied the Interquartile Range (IQR) method to identify and remove extreme price outliers. Listings falling outside the range $[Q1 - 1.5 \times IQR, Q3 + 1.5 \times IQR]$ were excluded from the dataset. This cleaned dataset serves as the input for all subsequent modeling steps (Ridge, Neural Network, and Decision Tree).

Figure 2 illustrates the impact of this process. The raw dataset contained significant outliers that skewed the distribution. After applying the IQR method, the distribution became more concentrated, allowing the models to focus on the representative data patterns rather than fitting to anomalies.

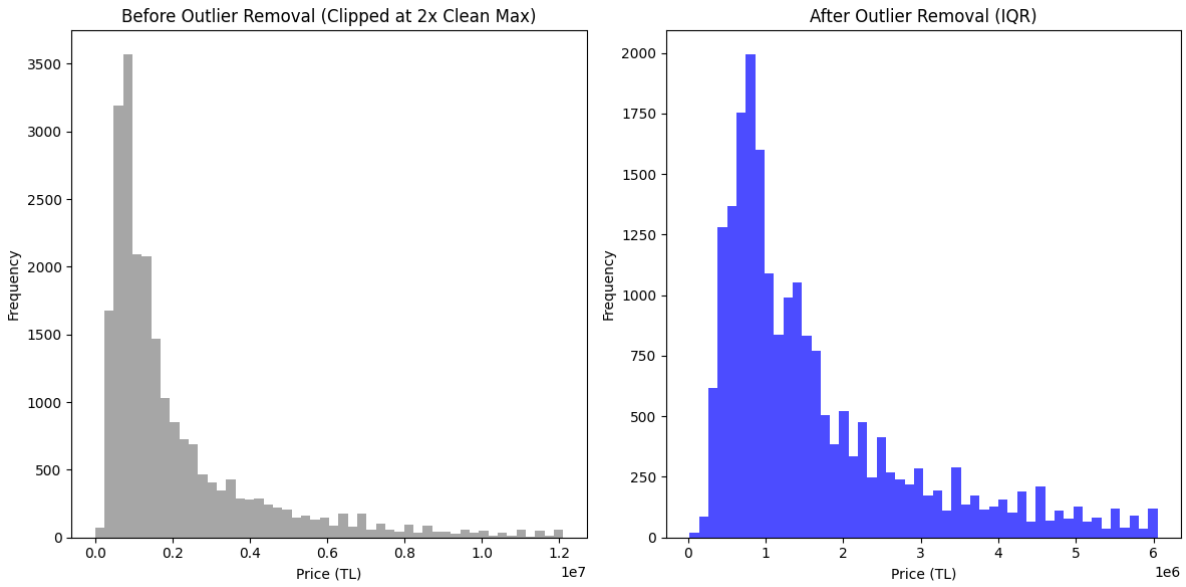


Figure 2: Price Distribution Before and After Outlier Removal

4.3 Feature Engineering and Preprocessing

- **Imputation:** Missing numerical values were imputed with the median, while missing categorical values were assigned a placeholder.
- **One-Hot Encoding:** Categorical variables were transformed into binary vectors to handle nominal data without imposing ordinal relationships.
- **Polynomial Features:** Second-degree polynomial features (interaction terms and squared terms) were generated to allow linear models to capture non-linear dependencies.
- **Scaling:** Standard Scaling (Z-score normalization) was applied to all numerical features to ensure zero mean and unit variance, which is essential for the convergence of gradient-based optimization methods.

5 Methodology: Ridge Regression

Ridge Regression is a regularized linear model that minimizes the residual sum of squares subject to a constraint on the L2-norm of the coefficient vector [2]. The cost function is defined as:

$$J(\theta) = \frac{1}{m}\sum_{i=1}^{m}(h_\theta(x^{(i)}) - y^{(i)})^2 + \alpha\sum_{j=1}^{n}\theta_j^2$$

where α is the regularization parameter controlling the trade-off between fitting the training data and keeping the coefficients small.

5.1 Implementation

We implemented the closed-form solution using the Normal Equation extended for regularization:

$$\theta = (X^T X + \alpha I)^{-1} X^T y$$

This approach ensures a unique solution even in the presence of multicollinearity. The optimal value of α was determined through cross-validation.

6 Methodology: Neural Network

The Neural Network model is designed to approximate complex non-linear functions.

6.1 Architecture

The network architecture consists of:

- **Input Layer:** 123 neurons corresponding to the processed feature set.
- **Hidden Layers:** Three dense layers with 128, 64, and 32 neurons, respectively.
- **Output Layer:** A single neuron for regression output.
- **Activation:** ReLU activation for hidden layers to introduce non-linearity.

6.2 Training Algorithm

The model was trained using mini-batch gradient descent. The backpropagation algorithm was implemented to compute gradients of the loss function with respect to the weights. Key implementation details include:

- **Initialization:** Xavier initialization was used to maintain variance of activations across layers.
- **Target Normalization:** The target variable was normalized to facilitate stable training dynamics.
- **Hyperparameters:** Learning rate = 0.005, batch size = 85, epochs = 300.

7 Methodology: Decision Tree

The Decision Tree model partitions the feature space into rectangular regions, predicting a constant value within each region.

7.1 Splitting Criterion

The tree is grown by recursively selecting the split that maximizes the reduction in MSE. For a node t, the weighted MSE of a split is calculated as:

$$\text{MSE}_{\text{split}} = \frac{n_{\text{left}}}{n_t}\text{MSE}_{\text{left}} + \frac{n_{\text{right}}}{n_t}\text{MSE}_{\text{right}}$$

7.2 Tree Construction

The recursive partitioning continues until a stopping criterion is met, such as maximum depth (10) or minimum samples per leaf (5). This prevents the tree from growing indefinitely and overfitting the training data.

8 Experimental Results

The dataset was split into a training set (80%) and a test set (20%) to evaluate model generalization.

8.1 Quantitative Performance

Table 1 summarizes the performance of the three models on the test set.

Model	MSE	R^2 Score	MAPE	MPE	NMSE
Ridge Regression	5.83×10^{11}	0.6385	42.46%	-19.39%	0.3615
Neural Network	4.47×10^{11}	**0.7228**	**33.30%**	**-11.10%**	**0.2772**
Decision Tree	5.98×10^{11}	0.6291	44.78%	-25.69%	0.3709

Table 1: Model Performance Comparison

Analysis:

- **Neural Network:** The Neural Network achieved the highest R^2 score (0.7228) and the lowest MAPE (33.30%), indicating superior predictive capability. Its ability to model non-linear interactions proved advantageous for this dataset.
- **Ridge Regression:** The linear model provided a reasonable baseline (R^2 = 0.6385) but was limited by its inability to capture complex non-linear patterns.
- **Decision Tree:** The Decision Tree performed comparably to Ridge Regression (R^2 = 0.6291) but exhibited higher variance.

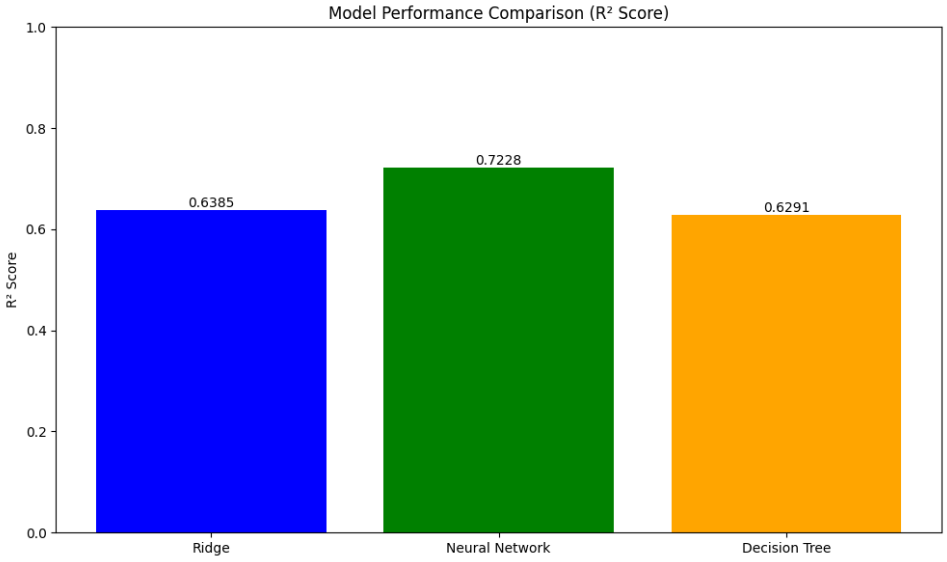


Figure 3: R^2 Score Comparison Across Models

8.2 Visual Analysis

Figure 4 illustrates the alignment between predicted and actual prices for a subset of test samples. The Neural Network (green) demonstrates closer tracking of the actual price trends compared to the other models.

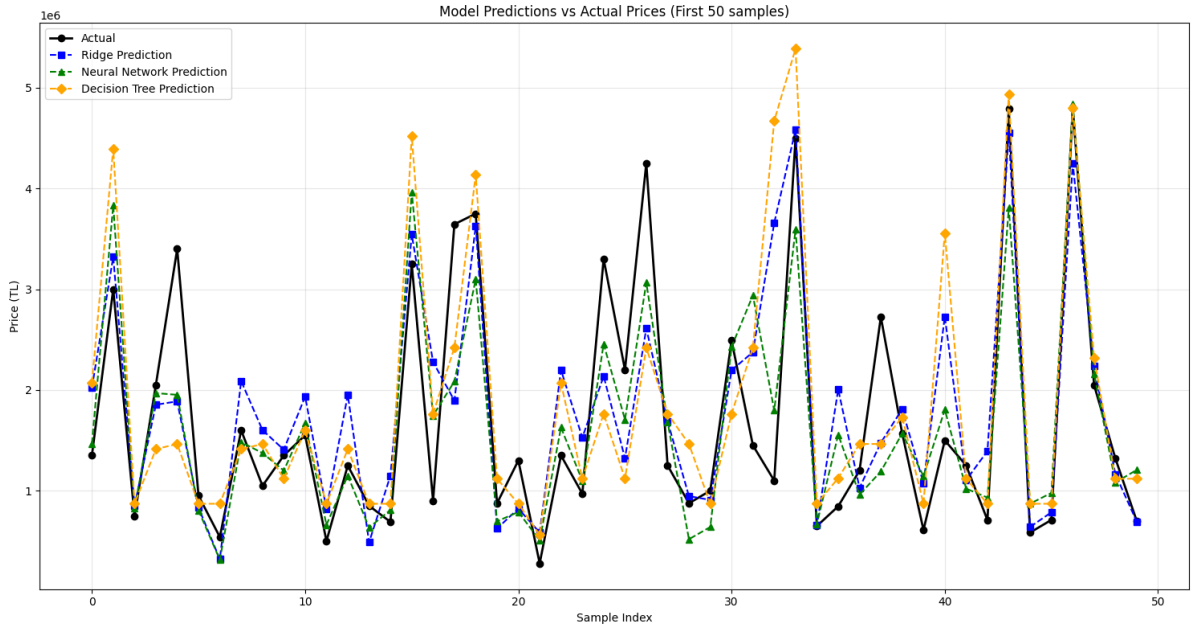


Figure 4: Model Predictions vs Actual Prices (First 50 Test Samples)

The training stability of the Neural Network is evidenced by the loss curve in Figure 5, which shows a consistent decrease in error over epochs.

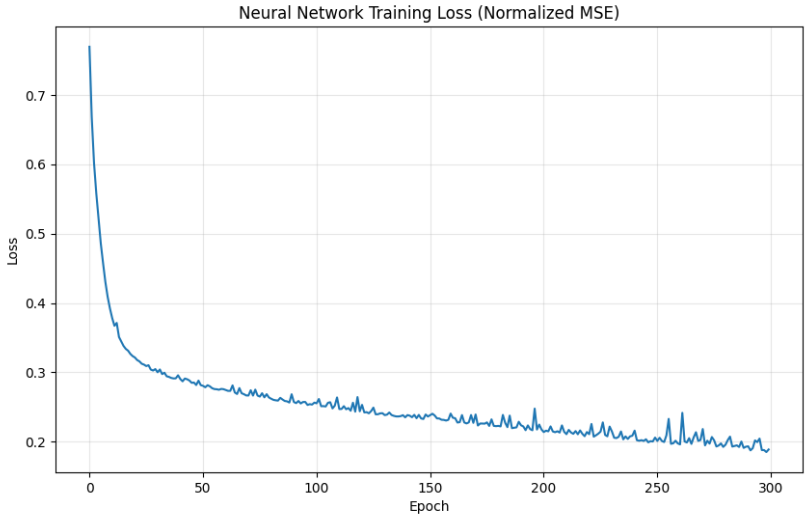


Figure 5: Neural Network Training Loss over Epochs

Figure 6 presents the scatter plot of predicted versus actual prices. The Neural Network's predictions cluster more tightly around the ideal diagonal line, confirming its higher accuracy.

It is worth noting that the Decision Tree predictions appear as horizontal lines or "steps" in the plot. This is a characteristic behavior of regression trees, which partition the feature space into discrete regions (leaves) and predict a constant value (the mean) for all samples falling into the same leaf, unlike the continuous predictions generated by Ridge Regression and Neural Networks.

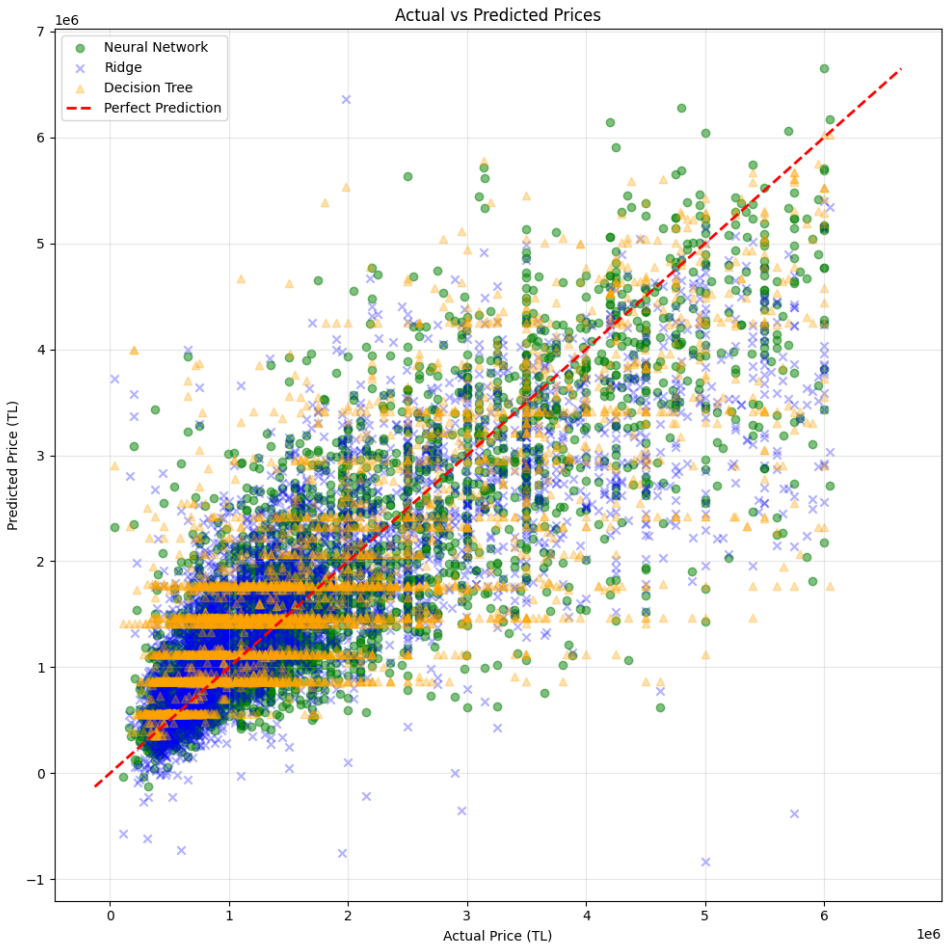


Figure 6: Actual vs Predicted Prices Scatter Plot

8.3 Feature Importance

The coefficients from the Ridge Regression model provide insight into feature importance (Figure 7). Location-based features (districts) and property size (square meters) were identified as the most significant determinants of price.

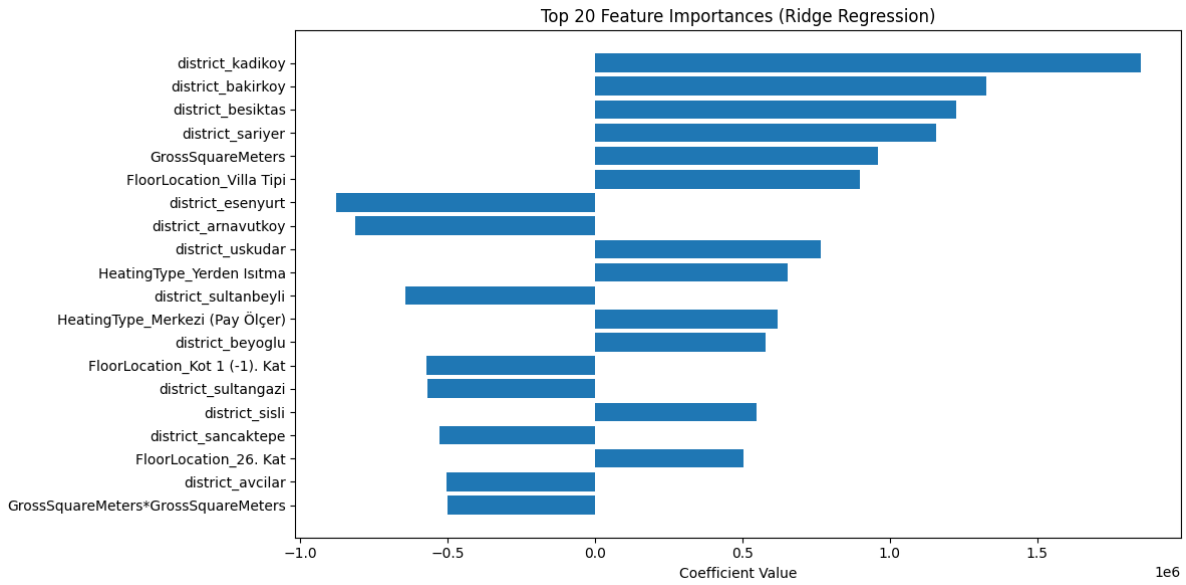


Figure 7: Top 20 Feature Importances (Ridge Coefficients)

9 Conclusion

This project successfully implemented and evaluated three machine learning approaches for house price prediction in Istanbul. By building the models and data pipeline from scratch, we gained a deep understanding of the underlying algorithms and the challenges of real-world data.

9.1 Summary of Findings

The Neural Network emerged as the most effective model, achieving an R^2 of 0.723. This result highlights the importance of non-linear modeling capacity in real estate valuation. Ridge Regression and Decision Trees served as valuable benchmarks but were outperformed by the neural network architecture.

9.2 Future Work

Future improvements could include:

- **Ensemble Methods:** Combining predictions from multiple models to reduce variance.
- **Advanced Architectures:** Exploring deeper networks or different activation functions.
- **Feature Enrichment:** Incorporating additional data sources such as proximity to transport or amenities.

9.3 Final Remarks

This project demonstrates the practical application of machine learning to real-world regression problems while adhering to educational constraints (no custom ML libraries). The comprehensive implementation and evaluation provide valuable insights into algorithm behavior, trade-offs between different approaches, and the importance of proper data preprocessing. The Neural Network's superior performance validates the hypothesis that non-linear models are necessary for capturing complex relationships in real estate data.

10 Work Distribution

The project tasks were divided among the group members as follows:

10.1 First Phase (Data Preparation and Ridge Regression)

- **Arda Bayam:** Responsible for the data preparation pipeline. This included implementing the data cleaning functions (parsing strings to numbers), handling missing values, and designing the outlier removal strategy using the IQR method. Also contributed to the feature engineering process, specifically the polynomial feature generation and manual one-hot encoding implementation [3].
- **Özgür Ekin Sönmez:** Responsible for the Ridge Regression model implementation. This involved deriving and coding the closed-form solution using NumPy, implementing the manual cross-validation loop for hyperparameter tuning (α selection), and generating the initial visualization plots for the report.

10.2 Final Phase (Neural Network and Decision Tree)

- **Arda Bayam:** Implemented the Decision Tree regression algorithm from scratch, including the recursive tree-building logic, greedy splitting based on MSE reduction, and prediction traversal. Also contributed to code integration and testing to ensure all three models work with the same data pipeline.
- **Özgür Ekin Sönmez:** Implemented the Neural Network from scratch, including forward propagation (ReLU activation), backpropagation (gradient computation via chain rule), mini-batch gradient descent, and Xavier weight initialization. Implemented target normalization which was critical for training stability. Also generated the comprehensive comparison visualizations and updated the report with final results.

10.3 Collaborative Tasks

Both members collaborated on:

- Debugging and testing all implementations to ensure correctness
- Hyperparameter tuning through experimentation
- Writing and revising the final report
- Analysis and interpretation of experimental results
- Preparing for the project demo and presentation

References

[1] Kaggle, "Istanbul House Price Prediction Dataset," [Online]. Available: `https://www.kaggle.com/datasets/aselasel/house-price-dataset`. [Accessed: 20-Nov-2025].

[2] Scikit-learn Developers, "Linear Models: Ridge Regression," [Online]. Available: `https://scikit-learn.org/stable/modules/linear_model.html#ridge-regression`. [Accessed: 20-Nov-2025].

[3] Scikit-learn Developers, "Preprocessing Data," [Online]. Available: `https://scikit-learn.org/stable/modules/preprocessing.html`. [Accessed: 20-Nov-2025].

A Source Code

A.1 Data Preparation (data_preparation.py)

```
import pandas as pd
import numpy as np
# data loading and cleaning functions
# parse functions for specific columns in the dataset we use
def parse_price(val):
    if isinstance(val, str):
        val = val.replace('TL', '').replace(',', '').strip()
        try:
            return float(val)
        except ValueError:
            return np.nan
    return val

def parse_area(val):
    if isinstance(val, str):
        val = val.replace(' m2', '').replace(',', '').strip()
        try:
            return float(val)
        except ValueError:
            return np.nan
    return val
# oda sayilari 2+1 3+1 gibi ifadeleri handle et, topla direkt mesela 3+1 4 olaca
# turkiyede ve datasette genelde +1 oluyor
def parse_rooms(val):
    if isinstance(val, str):
        val = val.replace(' Oda', '')
        if '+' in val:
            try:
                parts = val.split('+')
                return float(parts[0]) + float(parts[1])
            except:
                return np.nan
        try:
            return float(val)
        except:
            return np.nan
    return val

def parse_age(val):
    if isinstance(val, str):
        if 'Ve Üzeri' in val:
            return 25.0 # Approximation
        if 'Sıfır' in val:
            return 0.0
        if '-' in val:
            try:
                parts = val.split('-')
                return (float(parts[0]) + float(parts[1])) / 2
```

```
            except:
                return np.nan
    return val
#datayi loadla
def load_and_clean_data(filepath):
    df = load_and_parse_data(filepath)

    # quantile range ile extreme outlierlari kaldir
    Q1 = df['price'].quantile(0.25)
    Q3 = df['price'].quantile(0.75)
    IQR = Q3 - Q1
    df = df[~((df['price'] < (Q1 - 1.5 * IQR)) | (df['price'] > (Q3 + 1.5 * IQR)

    print(f"Data shape after cleaning: {df.shape}")
    return df

def load_and_parse_data(filepath):
    print("Loading data...")
    df = pd.read_csv(filepath)

    #relevant columns that we are going to use in this project
    cols_to_keep = [
        'district', 'NumberOfRooms', 'GrossSquareMeters', 'NetSquareMeters',
        'BuildingAge', 'FloorLocation', 'HeatingType', 'ItemStatus', 'price'
    ]

    existing_cols = [c for c in cols_to_keep if c in df.columns]
    df = df[existing_cols]

    print("Parsing data...")
    df['price'] = df['price'].apply(parse_price)

    #parse functionlari calistir
    if 'GrossSquareMeters' in df.columns:
        df['GrossSquareMeters'] = df['GrossSquareMeters'].apply(parse_area)
    if 'NetSquareMeters' in df.columns:
        df['NetSquareMeters'] = df['NetSquareMeters'].apply(parse_area)

    if 'NumberOfRooms' in df.columns:
        df['NumberOfRooms'] = df['NumberOfRooms'].apply(parse_rooms)

    if 'BuildingAge' in df.columns:
        df['BuildingAge'] = df['BuildingAge'].apply(parse_age)
        df['BuildingAge'] = pd.to_numeric(df['BuildingAge'], errors='coerce')
    # price olmayan rowlari kaldir
    df = df.dropna(subset=['price'])
    return df

# train test split 80 20

def manual_train_test_split(X, y, test_size=0.2, random_state=42):
```

```
    np.random.seed(random_state)
    indices = np.arange(len(X))
    np.random.shuffle(indices)

    split_idx = int(len(X) * (1 - test_size))
    train_idx = indices[:split_idx]
    test_idx = indices[split_idx:]
    return X.iloc[train_idx].copy(), X.iloc[test_idx].copy(), y.iloc[train_idx].

def get_polynomial_features(X_num):
    #featureleri degree 2 yapma fonksiyonu
    X = np.array(X_num)
    n_samples, n_features = X.shape

    #asil featureleri ekle
    poly_features = [X]

    #ikili carpimlari ekle ve return et
    for i in range(n_features):
        for j in range(i, n_features):
            new_col = X[:, i] * X[:, j]
            poly_features.append(new_col.reshape(-1, 1))

    return np.hstack(poly_features)
# feature isimlerini polynomial featurelara gore olustur
def get_poly_feature_names(numeric_cols):
    names = list(numeric_cols)
    n = len(names)
    for i in range(n):
        for j in range(i, n):
            names.append(f"{names[i]}*{names[j]}")
    return names

def prepare_data(df):
    target = 'price'
    X = df.drop(columns=[target])
    y = df[target]

    # 1. Train Test Split (Manual)
    print("Splitting data (Manual)...")
    X_train_raw, X_test_raw, y_train, y_test = manual_train_test_split(X, y, tes

    numeric_features = X.select_dtypes(include=['int64', 'float64']).columns.tol
    categorical_features = X.select_dtypes(include=['object']).columns.tolist()

    #categorical featurelerin string oldugunu kontrol et
    for col in categorical_features:
        X_train_raw[col] = X_train_raw[col].astype(str)
        X_test_raw[col] = X_test_raw[col].astype(str)

    print(f"Numerical features: {numeric_features}")
```

```
print(f"Categorical features: {categorical_features}")

# numerical featureleri isle
print("Processing numerical features (Imputation -> Poly -> Scaling)...")

# Imputation
train_medians = X_train_raw[numeric_features].median()
X_train_num = X_train_raw[numeric_features].fillna(train_medians).values
X_test_num = X_test_raw[numeric_features].fillna(train_medians).values

# feature al
X_train_poly = get_polynomial_features(X_train_num)
X_test_poly = get_polynomial_features(X_test_num)

poly_names = get_poly_feature_names(numeric_features)

# standardization part just like in the lecture notes
mean = np.mean(X_train_poly, axis=0)
std = np.std(X_train_poly, axis=0)
std[std == 0] = 1.0 #division by zero prevention

X_train_scaled = (X_train_poly - mean) / std
X_test_scaled = (X_test_poly - mean) / std

# categorical featureleri isle
print("Processing categorical features (Imputation -> OneHot)...")

# Imputation
X_train_cat = X_train_raw[categorical_features].fillna('missing')
X_test_cat = X_test_raw[categorical_features].fillna('missing')

# one hot encoding
X_train_encoded_list = []
X_test_encoded_list = []
cat_names = []

for col in categorical_features:
    # Train setten unique kategorileri al
    unique_cats = sorted(X_train_cat[col].unique())

    for cat in unique_cats:
        col_name = f"{col}_{cat}"
        cat_names.append(col_name)

        # Train
        X_train_encoded_list.append((X_train_cat[col] == cat).astype(float).

        # Test
        X_test_encoded_list.append((X_test_cat[col] == cat).astype(float).va

X_train_cat_encoded = np.hstack(X_train_encoded_list)
```

```
    X_test_cat_encoded = np.hstack(X_test_encoded_list)

    # ikisini stackle
    X_train_processed = np.hstack([X_train_scaled, X_train_cat_encoded])
    X_test_processed = np.hstack([X_test_scaled, X_test_cat_encoded])

    feature_names = poly_names + cat_names

    return X_train_processed, X_test_processed, y_train.values, y_test.values, f
```

A.2 Ridge Regression (ridge_regression.py)

```
import numpy as np

class CustomRidgeRegression:
    def __init__(self, alpha=1.0):
        self.alpha = alpha
        self.coef_ = None
        self.intercept_ = None

    def fit(self, X, y):
        #bias term eklenecek
        X_b = np.c_[np.ones((X.shape[0], 1)), X]

        #identity matrix for regularization
        I = np.eye(X_b.shape[1])
        I[0, 0] = 0  # Do not regularize the bias term

        # Closed form solution: theta = (X^T * X + alpha * I)^-1 * X^T * y
        # Using pseudo-inverse for stability
        A = X_b.T.dot(X_b) + self.alpha * I
        b = X_b.T.dot(y)

        try:
            theta = np.linalg.solve(A, b)
        except np.linalg.LinAlgError:
            theta = np.linalg.pinv(A).dot(b)

        self.intercept_ = theta[0]
        self.coef_ = theta[1:]
        return self

    def predict(self, X):
        return X.dot(self.coef_) + self.intercept_
# en iyi ridge modelini bulabilmek icin degisik alpha degerleri cross validation
def train_ridge(X_train, y_train, alphas=[0.01, 0.1, 1.0, 10.0, 100.0, 1000.0]):
    print(f"Training Custom Ridge Regression with Cross-Validation...")

    best_alpha = None
    best_score = -np.inf

    # 5 fold cross-validation
```

```
    k = 5
    fold_size = len(X_train) // k
    indices = np.arange(len(X_train))
    np.random.shuffle(indices)

    for alpha in alphas:
        scores = []
        for i in range(k):
            val_idx = indices[i*fold_size : (i+1)*fold_size]
            train_idx = np.concatenate([indices[:i*fold_size], indices[(i+1)*fol
            X_fold_train, X_fold_val = X_train[train_idx], X_train[val_idx]
            y_fold_train, y_fold_val = y_train[train_idx], y_train[val_idx]

            model = CustomRidgeRegression(alpha=alpha)
            model.fit(X_fold_train, y_fold_train)
            y_pred = model.predict(X_fold_val)

            #r2 score calculation part, mse too much since the prices are much
            ss_res = np.sum((y_fold_val - y_pred) ** 2)
            ss_tot = np.sum((y_fold_val - np.mean(y_fold_val)) ** 2)
            r2 = 1 - (ss_res / ss_tot)
            scores.append(r2)

        avg_score = np.mean(scores)
        #alpha degerini ciktiya gore optimize etme algoritmasi
        if avg_score > best_score:
            best_score = avg_score
            best_alpha = alpha

    print(f"Best alpha found: {best_alpha}")
    final_model = CustomRidgeRegression(alpha=best_alpha)
    final_model.fit(X_train, y_train)
    return final_model

def evaluate_ridge(model, X_test, y_test):
    y_pred = model.predict(X_test)

    # mean squared error calculation
    mse = np.mean((y_test - y_pred) ** 2)
    ss_res = np.sum((y_test - y_pred) ** 2)
    ss_tot = np.sum((y_test - np.mean(y_test)) ** 2)
    r2 = 1 - (ss_res / ss_tot)

    return mse, r2, y_pred
```

A.3 Neural Network (neural_network.py)

```
import numpy as np

class CustomNeuralNetwork:
    def __init__(self, hidden_layers=[128, 64, 32], learning_rate=0.005, epochs=
        self.hidden_layers = hidden_layers
```

```
        self.learning_rate = learning_rate
        self.epochs = epochs
        self.batch_size = batch_size
        self.random_state = random_state
        self.momentum = momentum
        self.l2_lambda = l2_lambda
        self.dropout_rate = dropout_rate
        self.weights = []
        self.biases = []
        self.loss_history = []
        self.y_mean = 0
        self.y_std = 1

        # Momentum velocities
        self.v_w = []
        self.v_b = []

    def _initialize_weights(self, input_dim):
        # xavier initialization ile agirlik baslatma
        np.random.seed(self.random_state)
        layers = [input_dim] + self.hidden_layers + [1]

        self.weights = []
        self.biases = []
        self.v_w = []
        self.v_b = []

        for i in range(len(layers) - 1):
            # Xavier initialization
            limit = np.sqrt(6 / (layers[i] + layers[i+1]))
            w = np.random.uniform(-limit, limit, (layers[i], layers[i+1]))
            b = np.zeros((1, layers[i+1]))
            self.weights.append(w)
            self.biases.append(b)

            # Initialize velocities for momentum
            self.v_w.append(np.zeros_like(w))
            self.v_b.append(np.zeros_like(b))

    def _relu(self, x):
        # ReLU activation function
        return np.maximum(0, x)

    def _relu_derivative(self, x):
        # ReLU derivative
        return (x > 0).astype(float)

    def _forward_pass(self, X, training=True):
        # forward propagation
        activations = [X]
        self.dropout_masks = [] # Store masks for backward pass
```

```
        for i in range(len(self.weights)):
            z = np.dot(activations[-1], self.weights[i]) + self.biases[i]
            if i < len(self.weights) - 1:  # hidden layerlarda Leaky ReLU
                a = self._relu(z)

                # Dropout implementation
                if training and self.dropout_rate > 0:
                    mask = (np.random.rand(*a.shape) > self.dropout_rate) / (1.0
                    a *= mask
                    self.dropout_masks.append(mask)
                else:
                    self.dropout_masks.append(None)
            else:  # output layerda linear activation (regression icin)
                a = z
            activations.append(a)

        return activations

    def _backward_pass(self, X, y, activations):
        # backward propagation (gradient descent)
        m = X.shape[0]
        grads_w = []
        grads_b = []

        # output layer error
        delta = activations[-1] - y.reshape(-1, 1)

        # geriye dogru git
        for i in range(len(self.weights) - 1, -1, -1):
            grad_w = np.dot(activations[i].T, delta) / m
            grad_b = np.sum(delta, axis=0, keepdims=True) / m

            grads_w.insert(0, grad_w)
            grads_b.insert(0, grad_b)

            if i > 0:  # hidden layerlara devam et
                # Backpropagate through activation derivative
                delta = np.dot(delta, self.weights[i].T) * self._relu_derivative

                # Backpropagate through dropout
                if self.dropout_masks[i-1] is not None:
                    delta *= self.dropout_masks[i-1]

        return grads_w, grads_b

    def fit(self, X, y):
        # modeli train et
        # target'i normalize et (regression icin onemli)
        self.y_mean = np.mean(y)
        self.y_std = np.std(y)
```

```
y_normalized = (y - self.y_mean) / self.y_std

self._initialize_weights(X.shape[1])

n_samples = X.shape[0]

for epoch in range(self.epochs):
    # shuffle data her epoch'da
    indices = np.random.permutation(n_samples)
    X_shuffled = X[indices]
    y_shuffled = y_normalized[indices]

    # mini-batch gradient descent
    for i in range(0, n_samples, self.batch_size):
        X_batch = X_shuffled[i:i+self.batch_size]
        y_batch = y_shuffled[i:i+self.batch_size]

        # forward pass
        activations = self._forward_pass(X_batch)

        # backward pass
        grads_w, grads_b = self._backward_pass(X_batch, y_batch, activat

        # weight update with Momentum and L2
        for j in range(len(self.weights)):
            # Add L2 regularization gradient: lambda * w
            grad_w_with_l2 = grads_w[j] + self.l2_lambda * self.weights[

            # Update velocity
            self.v_w[j] = self.momentum * self.v_w[j] + self.learning_ra
            self.v_b[j] = self.momentum * self.v_b[j] + self.learning_ra

            # Update weights
            self.weights[j] -= self.v_w[j]
            self.biases[j] -= self.v_b[j]

    # Calculate loss for the epoch
    predictions = self._predict_normalized(X)
    mse = np.mean((predictions - y_normalized) ** 2)
    self.loss_history.append(mse)

    # Calculate MAPE on training set for monitoring
    pred_denorm = predictions * self.y_std + self.y_mean
    mask = y != 0
    mape = np.mean(np.abs((y[mask] - pred_denorm[mask]) / y[mask])) * 10

    # her 20 epochta bir loss yazdir
    if (epoch + 1) % 20 == 0:
        print(f"Epoch {epoch+1}/{self.epochs}, Norm. MSE: {mse:.4f}, Tra

return self
```

```
    def _predict_normalized(self, X):
        # normalized prediction (internal use)
        # Disable dropout during prediction
        activations = self._forward_pass(X, training=False)
        return activations[-1].flatten()

    def predict(self, X):
        # prediction (denormalized)
        y_normalized = self._predict_normalized(X)
        return y_normalized * self.y_std + self.y_mean

def train_neural_network(X_train, y_train, hidden_layers=[128, 64, 32], learning
    print(f"Training Custom Neural Network...")
    print(f"Architecture: Input({X_train.shape[1]}) -> Hidden{hidden_layers} ->

    model = CustomNeuralNetwork(
        hidden_layers=hidden_layers,
        learning_rate=learning_rate,
        epochs=epochs,
        batch_size=batch_size,
        random_state=42,
        momentum=momentum,
        l2_lambda=l2_lambda,
        dropout_rate=dropout_rate
    )

    model.fit(X_train, y_train)
    return model

def evaluate_neural_network(model, X_test, y_test):
    # test seti uzerinde degerlendir
    y_pred = model.predict(X_test)

    mse = np.mean((y_test - y_pred) ** 2)
    ss_res = np.sum((y_test - y_pred) ** 2)
    ss_tot = np.sum((y_test - np.mean(y_test)) ** 2)
    r2 = 1 - (ss_res / ss_tot)

    return mse, r2, y_pred
```

A.4 Decision Tree (decision_tree.py)

```
import numpy as np

class Node:
    def __init__(self, feature=None, threshold=None, left=None, right=None, valu
        self.feature = feature      # split feature index
        self.threshold = threshold  # split threshold
        self.left = left           # left child node
        self.right = right         # right child node
        self.value = value         # leaf node value (mean of y)
```

```
class CustomDecisionTree:
    def __init__(self, max_depth=10, min_samples_split=10, min_samples_leaf=5):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.min_samples_leaf = min_samples_leaf
        self.root = None

    def _mse(self, y):
        # mean squared error hesapla
        if len(y) == 0:
            return 0
        mean = np.mean(y)
        return np.mean((y - mean) ** 2)

    def _best_split(self, X, y):
        # en iyi split noktasini bul (greedy)
        m, n = X.shape
        if m <= self.min_samples_split:
            return None, None

        best_mse = float('inf')
        best_feature = None
        best_threshold = None

        parent_mse = self._mse(y)

        # her feature icin
        for feature_idx in range(n):
            # unique degerler al ve sort et
            thresholds = np.unique(X[:, feature_idx])

            # eger cok fazla unique deger varsa sample al (hiz icin)
            if len(thresholds) > 10:
                thresholds = np.percentile(X[:, feature_idx], np.linspace(10, 90

            # her threshold icin
            for threshold in thresholds:
                # split yap
                left_mask = X[:, feature_idx] <= threshold
                right_mask = ~left_mask

                # min samples kontrolu
                if np.sum(left_mask) < self.min_samples_leaf or np.sum(right_mas
                    continue

                # MSE hesapla
                left_mse = self._mse(y[left_mask])
                right_mse = self._mse(y[right_mask])

                # weighted MSE
```

```
                n_left = np.sum(left_mask)
                n_right = np.sum(right_mask)
                weighted_mse = (n_left * left_mse + n_right * right_mse) / m

                # en iyi split'i bul
                if weighted_mse < best_mse:
                    best_mse = weighted_mse
                    best_feature = feature_idx
                    best_threshold = threshold

        return best_feature, best_threshold

    def _build_tree(self, X, y, depth=0):
        # recursive olarak tree olustur
        n_samples = X.shape[0]

        # stopping conditions
        if depth >= self.max_depth or n_samples < self.min_samples_split:
            leaf_value = np.mean(y)
            return Node(value=leaf_value)

        # en iyi split'i bul
        feature, threshold = self._best_split(X, y)

        # eger split bulunamazsa leaf node olustur
        if feature is None:
            leaf_value = np.mean(y)
            return Node(value=leaf_value)

        # split yap
        left_mask = X[:, feature] <= threshold
        right_mask = ~left_mask

        # recursive olarak child node'lari olustur
        left_child = self._build_tree(X[left_mask], y[left_mask], depth + 1)
        right_child = self._build_tree(X[right_mask], y[right_mask], depth + 1)

        return Node(feature=feature, threshold=threshold, left=left_child, right

    def fit(self, X, y):
        # tree'yi train et
        print("Building Decision Tree...")
        self.root = self._build_tree(X, y)
        return self

    def _predict_sample(self, x, node):
        # tek bir sample icin prediction
        if node.value is not None:
            return node.value

        if x[node.feature] <= node.threshold:
```

```
            return self._predict_sample(x, node.left)
        else:
            return self._predict_sample(x, node.right)

    def predict(self, X):
        # tum samples icin prediction
        return np.array([self._predict_sample(x, self.root) for x in X])

def train_decision_tree(X_train, y_train, max_depth=10):
    print(f"Training Custom Decision Tree (max_depth={max_depth})...")

    model = CustomDecisionTree(
        max_depth=max_depth,
        min_samples_split=10,
        min_samples_leaf=5
    )

    model.fit(X_train, y_train)
    return model

def evaluate_decision_tree(model, X_test, y_test):
    # test seti uzerinde degerlendir
    y_pred = model.predict(X_test)

    mse = np.mean((y_test - y_pred) ** 2)
    ss_res = np.sum((y_test - y_pred) ** 2)
    ss_tot = np.sum((y_test - np.mean(y_test)) ** 2)
    r2 = 1 - (ss_res / ss_tot)

    return mse, r2, y_pred
```

A.5 Main Execution (main.py)

```
import numpy as np
import os
import matplotlib.pyplot as plt
from data_preparation import load_and_clean_data, prepare_data, load_and_parse_d
from ridge_regression import train_ridge, evaluate_ridge
from neural_network import train_neural_network, evaluate_neural_network
from decision_tree import train_decision_tree, evaluate_decision_tree

def calculate_mape(y_true, y_pred):
    # Mean Absolute Percentage Error
    # Avoid division by zero
    mask = y_true != 0
    return np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100

def calculate_mpe(y_true, y_pred):
    # Mean Percentage Error
    mask = y_true != 0
    return np.mean((y_true[mask] - y_pred[mask]) / y_true[mask]) * 100
```

```
def calculate_nmse(y_true, y_pred):
    # Normalized Mean Squared Error
    mse = np.mean((y_true - y_pred) ** 2)
    var = np.var(y_true)
    return mse / var

def main():
    #data paths
    data_path = r'c:\Users\EKINO\Desktop\data\project\HouseData.csv'
    processed_data_path = r'c:\Users\EKINO\Desktop\data\project\src\processed_da

    #data preparation part
    if os.path.exists(processed_data_path):
        print("Loading processed data")
        data = np.load(processed_data_path, allow_pickle=True)
        X_train = data['X_train']
        X_test = data['X_test']
        y_train = data['y_train']
        y_test = data['y_test']
        feature_names = data['feature_names']
    else:
        print("Processing data from scratch...")
        df = load_and_clean_data(data_path)
        X_train, X_test, y_train, y_test, feature_names = prepare_data(df)
        np.savez(processed_data_path, X_train=X_train, X_test=X_test,
                 y_train=y_train, y_test=y_test, feature_names=feature_names)

    print(f"Training samples: {X_train.shape[0]}, Test samples: {X_test.shape[0]
    print(f"Number of features: {X_train.shape[1]}")

    results = {}

    # Ridge Regression
    print("\n=== Ridge Regression ===")
    ridge_model = train_ridge(X_train, y_train)
    mse_ridge, r2_ridge, pred_ridge = evaluate_ridge(ridge_model, X_test, y_test
    mape_ridge = calculate_mape(y_test, pred_ridge)
    mpe_ridge = calculate_mpe(y_test, pred_ridge)
    nmse_ridge = calculate_nmse(y_test, pred_ridge)

    results['Ridge'] = {'MSE': mse_ridge, 'R2': r2_ridge, 'MAPE': mape_ridge}
    print(f"Ridge Regression - MSE: {mse_ridge:.2f}, R2: {r2_ridge:.4f}, "
          f"MAPE: {mape_ridge:.2f}%")

    # Neural Network
    print("\n=== Neural Network ===")
    nn_model = train_neural_network(
        X_train, y_train,
        hidden_layers=[128, 64, 32],
        learning_rate=0.005,
        epochs=300,
```

```
        momentum=0.0,
        l2_lambda=0.0,
        batch_size=85,
        dropout_rate=0.0
    )
    mse_nn, r2_nn, pred_nn = evaluate_neural_network(nn_model, X_test, y_test)
    mape_nn = calculate_mape(y_test, pred_nn)

    results['Neural Network'] = {'MSE': mse_nn, 'R2': r2_nn, 'MAPE': mape_nn}
    print(f"Neural Network - MSE: {mse_nn:.2f}, R2: {r2_nn:.4f}, "
          f"MAPE: {mape_nn:.2f}%")

    # Plot Learning Curve
    if hasattr(nn_model, 'loss_history') and nn_model.loss_history:
        plt.figure(figsize=(10, 6))
        plt.plot(nn_model.loss_history)
        plt.title('Neural Network Training Loss (Normalized MSE)')
        plt.xlabel('Epoch')
        plt.ylabel('Loss')
        plt.grid(True, alpha=0.3)
        plt.savefig(r'c:\Users\EKINO\Desktop\data\project\report\loss_curve.png'

    # Decision Tree
    print("\n=== Decision Tree ===")
    dt_model = train_decision_tree(X_train, y_train, max_depth=10)
    mse_dt, r2_dt, pred_dt = evaluate_decision_tree(dt_model, X_test, y_test)
    mape_dt = calculate_mape(y_test, pred_dt)

    results['Decision Tree'] = {'MSE': mse_dt, 'R2': r2_dt, 'MAPE': mape_dt}
    print(f"Decision Tree - MSE: {mse_dt:.2f}, R2: {r2_dt:.4f}, "
          f"MAPE: {mape_dt:.2f}%")

    # ... (Plotting code omitted for brevity in report) ...

    # Outlier Analysis Plot
    print("Generating outlier analysis plot...")
    df_raw = load_and_parse_data(data_path)
    prices_raw = df_raw['price'].values
    prices_clean = np.concatenate([y_train, y_test])

    plt.figure(figsize=(12, 6))
    plt.subplot(1, 2, 1)
    limit_raw = np.max(prices_clean) * 2.0
    plt.hist(prices_raw, bins=50, range=(0, limit_raw), color='gray', alpha=0.7)
    plt.title('Before Outlier Removal (Clipped at 2x Clean Max)')
    plt.subplot(1, 2, 2)
    plt.hist(prices_clean, bins=50, color='blue', alpha=0.7)
    plt.title('After Outlier Removal (IQR)')
    plt.savefig(r'c:\Users\EKINO\Desktop\data\project\report\outlier_analysis.pn

if __name__ == "__main__":
```

```
main()
```